

LeerIA: Plataforma Inteligente para el Estudio Asistido por Documentos

Proyecto Final — Introducción a la Inteligencia Artificial 2026-1

Simón Sloan García Villa
Universidad EAFIT

2026-1

Abstract

LeerIA es una plataforma web basada en inteligencia artificial que permite convertir documentos académicos en una experiencia interactiva de estudio. El sistema permite subir archivos, procesarlos, extraer fragmentos relevantes, responder preguntas mediante un asistente conversacional y generar materiales de apoyo como resúmenes, quizzes, flashcards y guiones de video. La solución integra procesamiento de lenguaje natural, recuperación de información, embeddings semánticos y modelos generativos de lenguaje dentro de una arquitectura web compuesta por un frontend en React, un backend en FastAPI y persistencia en Supabase. El objetivo principal es ayudar a estudiantes a estudiar documentos extensos de manera más eficiente, personalizada y contextual.

Contents

1	Planteamiento del Problema	3
2	Objetivo General	3
3	Metodología	3
3.1	Diagrama de Flujo del Proceso	6
4	Desarrollo	7
4.1	Descripción General de la Solución	7
4.2	Conceptos de Inteligencia Artificial Utilizados	7
4.2.1	Procesamiento de Lenguaje Natural	7
4.2.2	Recuperación de Información	7
4.2.3	Embeddings Semánticos	7
4.2.4	Modelos Generativos de Lenguaje	7
4.2.5	RAG: Retrieval-Augmented Generation	8
4.3	Arquitectura del Sistema	8
4.4	Tecnologías Utilizadas	8
4.5	Modelo de Datos	8
4.6	Backend	9
4.7	Frontend	10
4.8	Bot Conversacional de Apoyo	10
4.9	Renderizado de Matemáticas	10
5	Resultados	10
5.1	Funcionalidades Implementadas	10
5.2	Evidencia del Funcionamiento	11
5.3	Métricas Propuestas	11
5.4	Resultados Cualitativos	12
6	Discusión	12
7	Interfaz de Usuario	13
8	Conclusiones	14
9	Instrucciones de Instalación y Ejecución	14
9.1	Backend	14
9.2	Frontend	14
9.3	Variables de Entorno	14

1 Planteamiento del Problema

En contextos universitarios, los estudiantes suelen enfrentarse a grandes volúmenes de información en forma de PDFs, apuntes, presentaciones, lecturas y documentos técnicos. El proceso tradicional de estudio exige leer, subrayar, resumir, crear preguntas, identificar conceptos clave y preparar repasos de forma manual. Esta tarea puede ser lenta, repetitiva y poco personalizada.

Además, muchos estudiantes no solo necesitan leer el material, sino también interactuar con él: hacer preguntas, pedir explicaciones, generar ejercicios, construir flashcards y comprobar su comprensión antes de exámenes o sustentaciones. Sin embargo, las herramientas tradicionales de lectura de documentos no suelen ofrecer una experiencia integrada que combine carga de archivos, conversación contextual y generación automática de material de estudio.

El problema abordado por este proyecto es:

¿Cómo diseñar una plataforma inteligente que permita a un estudiante cargar documentos académicos, conversar con ellos y generar materiales de estudio personalizados usando técnicas de inteligencia artificial?

LeerIA surge como una solución a este problema. La aplicación permite al usuario organizar documentos por materias, crear conversaciones asociadas a cada materia, hacer preguntas sobre el contenido y generar materiales de estudio basados en los documentos procesados.

La motivación principal es mejorar la eficiencia del aprendizaje, reducir la carga manual de preparación académica y ofrecer una interfaz funcional que integre procesamiento documental, recuperación semántica y generación de lenguaje natural.

2 Objetivo General

Desarrollar una plataforma web inteligente llamada **LeerIA** que permita a los estudiantes cargar documentos académicos, procesarlos mediante técnicas de procesamiento de lenguaje natural y utilizar modelos generativos para responder preguntas y crear materiales de estudio como resúmenes, quizzes, flashcards y guiones de video.

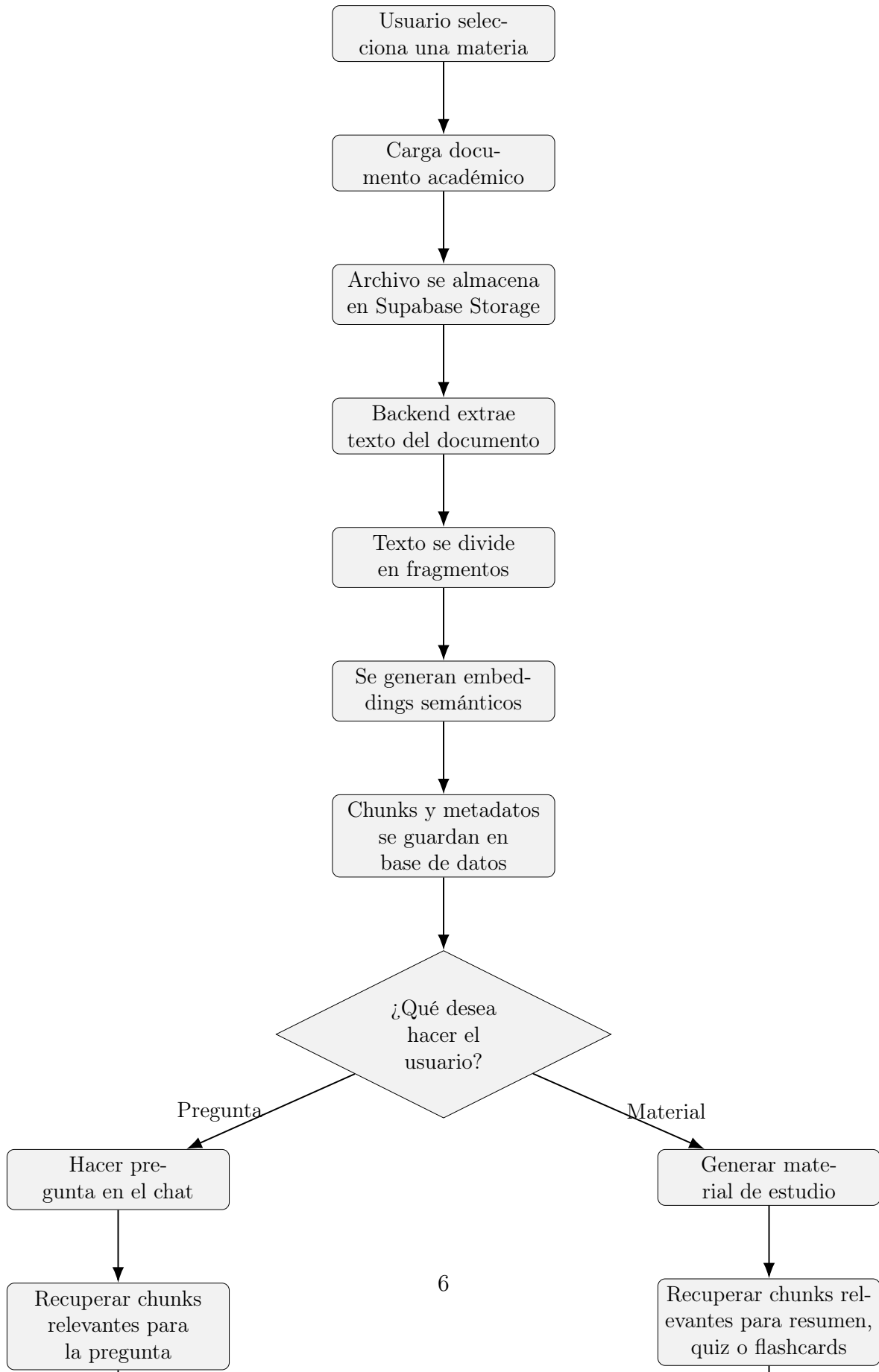
3 Metodología

La metodología del proyecto se basó en el desarrollo incremental de una aplicación web con componentes de inteligencia artificial. El proceso general se dividió en las siguientes fases:

1. **Definición del problema:** identificación de la necesidad de estudiar documentos académicos de forma más interactiva.
2. **Diseño de arquitectura:** separación entre frontend, backend, base de datos, almacenamiento y servicios de IA.
3. **Carga y gestión documental:** implementación de subida de archivos y organización por materias.

4. **Procesamiento de documentos:** extracción de texto, división en fragmentos y almacenamiento estructurado.
5. **Recuperación de información:** búsqueda de fragmentos relevantes según la pregunta o el tipo de material a generar.
6. **Generación con modelo de lenguaje:** uso de un modelo generativo para responder preguntas o crear materiales de estudio.
7. **Diseño de interfaz:** creación de una interfaz web funcional con panel de materias, conversaciones, documentos y herramientas de estudio.
8. **Pruebas funcionales:** validación manual del flujo completo desde carga de documento hasta generación de respuestas y materiales.

3.1 Diagrama de Flujo del Proceso



4 Desarrollo

4.1 Descripción General de la Solución

LeerIA se desarrolló como una aplicación web compuesta por tres capas principales:

- **Frontend:** interfaz de usuario construida con React y Vite.
- **Backend:** API REST construida con FastAPI.
- **Persistencia:** base de datos y almacenamiento de archivos mediante Supabase.

La aplicación permite organizar el estudio por materias. Cada materia puede tener documentos, conversaciones y materiales generados. El usuario puede seleccionar una materia, subir documentos, iniciar conversaciones y generar materiales de estudio.

4.2 Conceptos de Inteligencia Artificial Utilizados

El proyecto integra varios conceptos trabajados en el curso de Introducción a la Inteligencia Artificial:

4.2.1 Procesamiento de Lenguaje Natural

LeerIA procesa documentos académicos en lenguaje natural, extrae texto y permite al usuario hacer preguntas sobre el contenido. El sistema trabaja con texto no estructurado y lo transforma en fragmentos reutilizables para recuperación y generación.

4.2.2 Recuperación de Información

El sistema usa una estrategia de recuperación de fragmentos relevantes. Cuando el usuario hace una pregunta o solicita generar material, el backend busca los chunks más relacionados con la consulta. Esto permite que el modelo no responda únicamente desde conocimiento general, sino desde el contenido documental de la materia.

4.2.3 Embeddings Semánticos

Cada fragmento del documento se representa mediante embeddings, es decir, vectores numéricos que capturan información semántica del texto. Estos vectores permiten comparar la similitud entre una pregunta y los fragmentos almacenados.

4.2.4 Modelos Generativos de Lenguaje

LeerIA utiliza un modelo de lenguaje para generar respuestas, resúmenes, preguntas de quiz, flashcards y guiones de video. El modelo recibe como entrada el contexto documental recuperado y produce una salida en lenguaje natural o en JSON estructurado.

4.2.5 RAG: Retrieval-Augmented Generation

La técnica principal utilizada es *Retrieval-Augmented Generation* o RAG. Esta técnica combina recuperación de información con generación de texto. Primero se recuperan fragmentos relevantes de los documentos y luego se entregan como contexto al modelo generativo.

4.3 Arquitectura del Sistema

La arquitectura general de LeerIA se compone de los siguientes módulos:

- **Gestión de materias:** creación, edición y selección de materias.
- **Gestión de documentos:** subida, almacenamiento, procesamiento y listado de documentos.
- **Gestión de conversaciones:** creación de conversaciones por materia.
- **Chat RAG:** preguntas y respuestas basadas en documentos.
- **Generador de materiales:** creación de resúmenes, quizzes, flashcards y guiones de video.
- **Renderizado matemático:** visualización de fórmulas usando LaTeX y KaTeX.

4.4 Tecnologías Utilizadas

Tecnología	Uso dentro del proyecto
React + Vite	Construcción de la interfaz web.
TypeScript	Tipado del frontend y mayor robustez en componentes.
Tailwind CSS	Estilizado visual de la aplicación.
FastAPI	Construcción de la API REST del backend.
Supabase	Base de datos, almacenamiento de archivos y persistencia de entidades.
OpenAI API / Modelo generativo	Generación de respuestas y materiales de estudio.
Embeddings	Representación semántica de los fragmentos documentales.
KaTeX	Renderizado de fórmulas matemáticas en la interfaz.

Table 1: Tecnologías principales utilizadas en LeerIA.

4.5 Modelo de Datos

El sistema maneja varias entidades principales:

- **Subject:** representa una materia.
- **Document:** representa un archivo subido por el usuario.
- **DocumentChunk:** representa un fragmento procesado del documento.
- **Conversation:** representa una conversación asociada a una materia.
- **Message:** representa mensajes del usuario y del asistente.
- **GeneratedItem:** representa materiales generados como resumen, quiz, flashcards o guion.

Una decisión importante fue asociar los materiales generados no solo a una materia, sino también a una conversación mediante `conversation_id`. Esto evita que diferentes conversaciones dentro de una misma materia reutilicen siempre el mismo resumen, quiz o conjunto de flashcards.

4.6 Backend

El backend está construido con FastAPI y expone endpoints REST para:

- crear y consultar materias;
- subir y procesar documentos;
- crear y consultar conversaciones;
- enviar mensajes al chat;
- generar materiales de estudio;
- recuperar artefactos generados.

El flujo de generación de material sigue esta lógica:

1. el usuario solicita un tipo de material;
2. el frontend envía `subject_id`, `conversation_id`, tipo de material y parámetros de generación;
3. el backend recupera fragmentos relevantes de los documentos;
4. el modelo generativo produce una respuesta estructurada;
5. el resultado se guarda en la tabla `generated_items`;
6. el frontend renderiza el material en una vista especializada.

4.7 Frontend

El frontend incluye una interfaz dividida en tres zonas:

- **Sidebar izquierdo:** muestra materias y conversaciones.
- **Área central:** muestra chat, estado vacío, resúmenes, quizzes o flashcards.
- **Panel derecho:** muestra herramientas de estudio, documentos recientes y estado del sistema.

El usuario puede interactuar con LeerIA sin necesidad de usar consola ni conocer detalles técnicos del backend. Esta interfaz cumple con el requisito de interacción directa con el sistema.

4.8 Bot Conversacional de Apoyo

LeerIA incluye un bot conversacional integrado. Este bot permite hacer preguntas sobre los documentos cargados y recibir explicaciones en lenguaje natural. El bot utiliza los fragmentos recuperados como contexto para responder, lo cual reduce el riesgo de respuestas inventadas y mejora la trazabilidad hacia los documentos de la materia.

4.9 Renderizado de Matemáticas

Dado que LeerIA está orientado a documentos académicos, especialmente materias como cálculo, álgebra o estadística, se implementó soporte para expresiones matemáticas. Las fórmulas se generan en LaTeX y se renderizan en el frontend usando KaTeX.

Ejemplo de una fórmula que puede ser renderizada por el sistema:

$$\iiint_V \nabla \cdot \mathbf{F} dV = \iint_S \mathbf{F} \cdot \mathbf{n} dS$$

Esto permite que el sistema sea útil no solo para textos generales, sino también para documentos técnicos y matemáticos.

5 Resultados

5.1 Funcionalidades Implementadas

Hasta el momento, LeerIA cuenta con las siguientes funcionalidades funcionales:

- creación y visualización de materias;
- creación de conversaciones por materia;
- subida de documentos;
- procesamiento de documentos en chunks;

- chat con respuestas basadas en documentos;
- generación de resúmenes;
- generación de quizzes;
- generación de flashcards;
- visualización de fórmulas matemáticas en chat y materiales;
- separación de materiales por conversación mediante `conversation_id`;
- interfaz web funcional y navegable.

5.2 Evidencia del Funcionamiento

La aplicación permite ejecutar el siguiente flujo completo:

1. el usuario crea o selecciona una materia;
2. el usuario sube un documento PDF;
3. el sistema procesa el archivo y extrae fragmentos;
4. el usuario realiza preguntas en el chat;
5. el sistema recupera fragmentos relevantes;
6. el modelo genera una respuesta contextual;
7. el usuario puede generar resumen, quiz o flashcards;
8. los materiales generados se muestran en vistas especializadas.

5.3 Métricas Propuestas

Aunque el proyecto se centra principalmente en una solución funcional, se proponen las siguientes métricas para evaluar el desempeño del sistema:

Métrica	Descripción	Estado
Tiempo de procesamiento de documento	Tiempo desde la subida hasta la generación de chunks.	Por medir
Precisión de recuperación	Proporción de chunks recuperados que realmente son relevantes.	Por medir
Calidad de respuesta	Evaluación humana de claridad, fidelidad y utilidad.	Evaluación manual
Consistencia del material generado	Verificación de que resúmenes, quizzes y flashcards correspondan al documento.	Evaluación manual
Usabilidad de interfaz	Facilidad para completar el flujo de carga, pregunta y generación.	Validación funcional

Table 2: Métricas propuestas para la evaluación de LeerIA.

5.4 Resultados Cualitativos

Durante las pruebas funcionales se observó que el sistema puede generar materiales coherentes a partir de los documentos cargados. También se identificaron y corrigieron problemas importantes, como:

- reutilización incorrecta de materiales generados entre conversaciones;
- recuperación de contexto demasiado general dentro de una materia;
- renderizado incorrecto de fórmulas matemáticas;
- necesidad de asociar materiales a conversaciones específicas;
- necesidad de limpiar y reforzar prompts para producir LaTeX válido.

Estas correcciones mejoraron la calidad de la experiencia de usuario y la coherencia entre conversación, documento y material generado.

6 Discusión

LeerIA se relaciona con herramientas modernas de aprendizaje asistido por inteligencia artificial, especialmente sistemas basados en RAG y asistentes conversacionales sobre documentos. A diferencia de un chatbot general, LeerIA restringe sus respuestas al contenido documental de cada materia, lo que aumenta la pertinencia para el estudio académico.

El enfoque RAG ha sido ampliamente usado para complementar modelos generativos con información externa recuperada desde una base de conocimiento. En este proyecto, la base de conocimiento corresponde a los documentos cargados por el estudiante. Esta decisión permite adaptar el sistema a distintas materias sin necesidad de entrenar un modelo desde cero.

Comparado con una solución de resumen tradicional, LeerIA ofrece mayor interacción porque permite preguntar, generar quizzes, crear flashcards y navegar por conversaciones. Comparado con una simple interfaz de chat, LeerIA agrega organización por materias, documentos y materiales generados.

Sin embargo, el sistema también presenta limitaciones:

- la calidad de las respuestas depende de la calidad del texto extraído de los documentos;
- si los documentos tienen mala extracción OCR o símbolos matemáticos mal convertidos, el modelo puede recibir contexto imperfecto;
- las respuestas generadas pueden requerir validación humana;
- la evaluación cuantitativa del sistema aún debe fortalecerse con métricas formales;
- el sistema depende de servicios externos para el modelo generativo y posiblemente para embeddings.

Frente al estado del arte, LeerIA implementa una versión práctica de RAG aplicada al estudio universitario. No pretende competir con plataformas comerciales a gran escala, sino demostrar la integración funcional de conceptos de inteligencia artificial en un problema real, con una interfaz usable y un flujo completo de extremo a extremo.

7 Interfaz de Usuario

La interfaz de LeerIA fue diseñada como una aplicación web moderna e intuitiva. El usuario puede:

- crear materias;
- seleccionar conversaciones;
- subir documentos;
- escribir preguntas;
- recibir respuestas del asistente;
- generar materiales de estudio desde el panel lateral;
- visualizar resúmenes, quizzes y flashcards.

La interfaz cumple el requisito de interacción directa con el sistema, ya que el usuario no necesita ejecutar scripts ni manipular archivos manualmente. Todo el flujo se realiza desde la aplicación web.

8 Conclusiones

LeerIA demuestra que es posible integrar técnicas de inteligencia artificial en una herramienta educativa funcional para apoyar el estudio de documentos académicos. El sistema combina procesamiento de lenguaje natural, recuperación semántica, modelos generativos, RAG e interfaz web en una solución completa.

El proyecto evidencia el uso de conceptos fundamentales del curso, especialmente procesamiento de lenguaje natural, recuperación de información y modelos generativos. Además, incluye un bot conversacional de apoyo, lo cual aporta valor adicional a la experiencia de usuario.

Como trabajo futuro, se propone:

- implementar métricas cuantitativas de recuperación;
- agregar evaluación automática de respuestas;
- permitir selección explícita de documentos por conversación;
- mejorar el procesamiento de fórmulas matemáticas desde PDFs;
- agregar exportación de materiales generados;
- implementar historial de versiones de resúmenes, quizzes y flashcards.

9 Instrucciones de Instalación y Ejecución

9.1 Backend

```
cd Backend/LeerIA-backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
fastapi dev app/main.py
```

9.2 Frontend

```
cd Frontend/LeerIA-frontend
npm install
npm run dev
```

9.3 Variables de Entorno

El backend requiere variables de entorno para conectarse a Supabase y al proveedor del modelo generativo. Un ejemplo de archivo `.env` sería:

```
SUPABASE_URL=...  
SUPABASE_SERVICE_ROLE_KEY=...  
OPENAI_API_KEY=...
```

El frontend requiere una URL base para la API:

```
VITE_API_URL=http://127.0.0.1:8000/api/v1
```

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems.
- [3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.
- [4] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.